



LeetCode 1342: Number of Steps to Reduce a Number to Zero

1. Problem Title & Link

Title: LeetCode 1342: Number of Steps to Reduce a Number to Zero

Link: <https://leetcode.com/problems/number-of-steps-to-reduce-a-number-to-zero/>

2. Problem Statement (Short Summary)

Given an integer num:

- If it is even $\rightarrow$ divide by 2
- If it is odd $\rightarrow$ subtract 1

Count how many steps are needed to make it become 0.

3. Examples (Input $\rightarrow$ Output)

Example 1

Input:

num = 14

Output:

6

Explanation:

14 $\rightarrow$ 7

7 $\rightarrow$ 6

6 $\rightarrow$ 3

3 $\rightarrow$ 2

2 $\rightarrow$ 1

1 $\rightarrow$ 0

6 steps.

Example 2

Input:

num = 8

Output:

4

Example 3

Input:

num = 0

Output:

0



4. Constraints

$0 \leq \text{num} \leq 10^6$

5. Core Concept (Pattern / Topic)

Simulation / Bit Manipulation

6. Thought Process (Step-by-Step Explanation)

Brute Force

Follow exactly what the problem says.

Repeatedly:

even $\rightarrow$ divide by 2

odd $\rightarrow$ subtract 1

until number becomes zero.

Since number shrinks every step, solution is efficient.

7. Visual / Intuition Diagram (ASCII Diagram)

```
14
  
Even
  ↓
 7
  
Odd
  ↓
 6
  
Even
  ↓
 3
  
Odd
  ↓
 2
  
Even
  ↓
 1
  
Odd
  ↓
```



0

8. Pseudocode

```
steps = 0

while num != 0

    if even
        num = num / 2
    else
        num = num - 1

    steps++

return steps
```

9. Code Implementation

Python

```
class Solution:
    def numberOfSteps(self, num):
        steps = 0

        while num:
            if num % 2 == 0:
                num //= 2
            else:
                num -= 1

            steps += 1

        return steps
```

Java

```
class Solution {
    public int numberOfSteps(int num) {

        int steps = 0;

        while(num != 0) {

            if(num % 2 == 0)
```

```
        num /= 2;
    else
        num--;

    steps++;
}

return steps;
}
```

10. Time & Space Complexity

Time Complexity : $O(\log n)$

Space Complexity : $O(1)$

11. Common Mistakes / Edge Cases

num = 0

Should return 0 immediately.

12. Detailed Dry Run

Input:

num = 14

Step	num	Operation
1	14	/2
2	7	-1
3	6	/2
4	3	-1
5	2	/2
6	1	-1

Answer:

6

13. Common Use Cases

Process reduction

Binary operations

State transitions

Resource consumption simulations



14. Common Traps

Forgetting num = 0 case

Using floating-point division

15. Builds To

LC 1342

LC 191 - Number of 1 Bits

LC 338 - Counting Bits

LC 231 - Power of Two

16. Alternate Approaches + Comparison

Approach	Time	Space
Simulation	$O(\log n)$	$O(1)$
Bit Manipulation	$O(\log n)$	$O(1)$

17. Why This Solution Works

Each operation strictly reduces the number, guaranteeing termination at zero.

18. Variations / Follow-Up Questions

1. Count operations using bit manipulation.
2. Reduce to 1 instead of 0.
3. What if divide-by-3 is allowed?